

Proyecto integrador: **Desarrollo y despliegue de una API** (Pipeline CI/CD con Git, Docker y GitHub Actions)

Experiencia educativa: Despliegue de Software (grupo: LIS 601)
Licenciatura en Ingeniería de Software
Período feb.-jul. 2026

I. Presentación

Este proyecto integrador tiene como finalidad que desarrolles competencias reales de Ingeniería de Software en un contexto colaborativo, aplicando el ciclo completo de desarrollo: diseño, codificación, pruebas, contenedorización y despliegue automático mediante un pipeline CI/CD. En equipo construirán una API funcional que se integre con servicios externos, persista datos en una base de datos y se despliegue de forma automática usando GitHub Actions y Docker.

Competencias que desarrollarás:

- Diseño de arquitecturas de software orientadas a servicios (API REST)
- Trabajo colaborativo con control de versiones (Git / GitHub)
- Automatización de pruebas e integración continua (GitHub Actions)
- Contenedorización de aplicaciones (Docker)
- Documentación técnica de software

Formación del equipo. Los equipos se conformarán por 3 a 4 integrantes. La participación equitativa es obligatoria y será verificada mediante el **historial de commits** en GitHub. **Cada integrante debe registrar contribuciones técnicas verificables.**

Rol y responsabilidades principales:

- Líder técnico (Tech Lead). Coordina la arquitectura, revisa pull requests, gestiona ramas principales y resuelve conflictos de merging.
- Desarrollador de backend. Implementa los endpoints de la API, lógica de negocio e integración con APIs externas.
- Control de calidad (DevOps / QA). Configura Docker, el pipeline CI/CD en GitHub Actions y escribe pruebas automatizadas.
- Documentación (Data / Docs). Diseña el modelo de base de datos, escribe LEARNINGS.md y documenta los endpoints (README).

II. Fases del proyecto

El proyecto se desarrolla en cinco fases, siguiendo los principios y prácticas de Ingeniería de Software.

Fase I: Diseño y planeación del proyecto

Objetivo. Definir la idea de la API, su arquitectura y la distribución de responsabilidades del equipo, aplicando principios de Ingeniería de Software.

Actividades:

1. Identificar el problema que resolverá la API (dominio: clima, gastronomía, chat, etc.).
2. Definir los endpoints que tendrá la API usando la notación REST (método, ruta, parámetros, respuesta).
3. Seleccionar la API externa a consumir (OpenWeatherMap, Google Maps, etc.).
4. Elegir el motor de base de datos (MySQL o MongoDB) y diseñar el modelo de datos.
5. Crear el repositorio en GitHub y definir la estrategia de ramas (main / develop / feature-*).
6. Redactar el documento de especificación inicial (README.md con descripción, arquitectura y plan de trabajo).
7. Exponer la idea al docente para recibir retroalimentación sobre la viabilidad.

Entregable: README.md inicial con descripción de la API, diagrama de arquitectura, lista de endpoints planeados y modelo de datos.

Fase 2: Desarrollo de la API

Objetivo. Implementar los endpoints de la API aplicando buenas prácticas de codificación, control de versiones y revisión de código entre pares.

Actividades:

8. Configurar el entorno de desarrollo local (lenguaje/framework de elección: Node.js/Express, Python/FastAPI, Spring Boot).
9. Implementar la conexión con la base de datos y las migraciones/esquemas necesarios.
10. Desarrollar los endpoints definidos en la Fase I, incluyendo la llamada a la API externa.
11. Usar ramas de funcionalidad (feature branches) por cada nuevo endpoint o módulo.
12. Practicar el flujo: git pull → develop en rama propia → Pull Request → Code Review → merge.
13. Resolver conflictos de merging de forma ordenada, documentando la solución.
14. Probar manualmente los endpoints con Postman y documentar los resultados.

Entregable. Código fuente completo en el repositorio con historial de commits individuales, al menos un Pull Request por integrante revisado y aprobado.

Fase 3: Contenedorización con Docker

Objetivo. Empaquetar la API y la base de datos en contenedores Docker para garantizar portabilidad y reproducibilidad del entorno.

Actividades:

15. Crear el Dockerfile de la API (imagen base, dependencias, comando de arranque).
16. Crear el Dockerfile de la base de datos (o usar imagen oficial como mysql:8 / mongo:6).
17. Escribir el archivo docker-compose.yml que orqueste la API y la base de datos en una red interna.
18. Verificar que la aplicación funciona correctamente ejecutando: `docker compose up --build`.
19. Usar variables de entorno (.env) para manejar credenciales de la BD y claves de la API externa.
20. Documentar los comandos Docker en el README.md (cómo construir y levantar los contenedores).

Entregable. Dockerfile, docker-compose.yml y sesión de Postman que demuestre que la API funciona desde los contenedores.

Fase 4: Pipeline CI/CD con GitHub Actions

Objetivo. Automatizar el ciclo de integración y despliegue de la API mediante GitHub Actions, aplicando el concepto de entrega continua.

Actividades:

21. Crear pruebas automatizadas básicas para al menos 3 endpoints.
22. Configurar el workflow de CI (.github/workflows/ci.yml) que ejecute las pruebas al hacer push.
23. Ampliar el pipeline con un paso de construcción de la imagen Docker (docker build).
24. Añadir un paso de publicación en Docker Hub o GitHub Container Registry.
25. Verificar que el pipeline pasa en verde (todos los pasos exitosos) antes de hacer merge a main.
26. Documentar el pipeline en el APRENDIZAJES.md explicando cada paso del workflow.

Entregable. Archivo .github/workflows/ci.yml funcional, badge de estado del pipeline en el README.md, evidencia de al menos 3 ejecuciones exitosas.

Fase 5: Presentación y documentación final

Objetivo. Comunicar el trabajo desarrollado de forma profesional, demostrar el funcionamiento de la API y reflexionar sobre el aprendizaje obtenido.

Actividades:

27. Preparar una presentación de máximo 5 diapositivas (problema, solución, arquitectura, demo, aprendizajes).
28. Realizar una demo en vivo usando Postman para invocar los endpoints desde los contenedores Docker.
29. Presentar el pipeline de CI/CD en ejecución (mostrar la pestaña Actions en GitHub).
30. Redactar el archivo APRENDIZAJES.md con: descripción funcional, decisiones técnicas, retos enfrentados y aprendizajes.
31. Asegurarse de que todo el código final esté commiteado y el repositorio esté limpio (sin archivos .env en el repo).

Entregable. Presentación, demo funcional, APRENDIZAJES.md en el repositorio, repositorio final ordenado.

Resumen de actividades (Semanal)

Semana	Fase	Foco principal	Entregable clave
Sem1	Fase 1	Organización del equipo, diseño de la API, arquitectura	Idea definida y repositorio creado
Sem2	Fase 1	Diagrama de arquitectura, diseño de endpoints, README.md	README.md inicial aprobado
Sem3	Fase 2	Desarrollo de endpoints, integración con API externa	Al menos 2 endpoints funcionando
Sem4	Fase 2	Endpoints completos, PRs revisados, pruebas con Postman	API funcional + commits de todos
Sem5	Fases 2-3	Docker: Dockerfile, docker-compose, variables de entorno	Contenedores corriendo localmente
Sem6	Fases 4-5	Pruebas automáticas, GitHub Actions, inicio de documentación	Pipeline CI/CD en verde
Sem7	Fase 5	Demo, presentación, APRENDIZAJES.md, limpieza del repo	Entrega y presentación final

Recomendaciones para cumplir el cronograma

- Define la idea con claridad antes de escribir código. Un buen diseño evita retrabajo.
- Haz `git pull` antes de cada sesión de trabajo. Los conflictos crecen si se ignoran.
- Cada integrante debe tener al menos un Pull Request aprobado antes de la sem5.
- Empieza Docker temprano. Los problemas de entorno son frecuentes la primera vez.
- El pipeline CI/CD debe pasar en verde antes de hacer merge a main.
- Ensaya la demo con Postman al menos 2 veces. La presentación se califica en vivo.

Señales de alerta que requieren acción inmediata:

- Sem3 y aún no hay un solo endpoint funcionando → revisar avance con el profesor.
- Sem5 y Docker no levanta → priorizar sobre cualquier otra tarea.
- Sem6 y el pipeline tiene errores → no agregar funcionalidad nueva hasta resolverlo.
- Algún integrante sin commits en la sem4 → redistribuir tareas de inmediato.

III. Requisitos técnicos

Todos los proyectos deben cumplir los siguientes requisitos para ser evaluados:

- ☐ La API consume al menos una API externa (OpenWeatherMap, Google Maps, etc.).
- ☐ Los datos se persisten en una base de datos (MySQL o MongoDB).
- ☐ La API y la base de datos se ejecutan en contenedores Docker independientes.
- ☐ El repositorio incluye un archivo docker-compose.yml funcional.
- ☐ El repositorio incluye al menos un workflow de GitHub Actions (.github/workflows/).
- ☐ Cada integrante tiene al menos 4 commits significativos en el historial.
- ☐ El README.md documenta cómo levantar el proyecto localmente.
- ☐ El archivo APRENDIZAJES.md describe el proyecto y los aprendizajes.

Importante: Seguridad del código

NUNCA subas al repositorio archivos con credenciales: .env, API keys, contraseñas.

Usa un archivo .env.example con las variables requeridas (sin valores reales) y añade .env al .gitignore.

IV. Estrategia de control de versiones

4.1 Flujo de trabajo con Git. El equipo trabajará con el siguiente modelo de ramas:

Rama	Propósito
main	Código estable y funcional. Solo se fusiona desde develop tras aprobación.
develop	Rama de integración continua. Recibe los Pull Requests de las ramas de funcionalidad.
feature/nombre	Rama individual por funcionalidad nueva (ej: feature/endpoint-clima). Se crea desde develop.
fix/nombre	Correcciones de errores. Se crea desde develop o main según urgencia.

4.2 Comandos Git esenciales

Practica el siguiente flujo de trabajo de manera constante:

```
git checkout develop && git pull origin develop    # sincroniza develop
git checkout -b feature/mi-endpoint                # crea tu rama
# ... desarrollas y haces commits ...
git push origin feature/mi-endpoint                # sube tu rama
# Crea un Pull Request en GitHub hacia develop
# Espera la revisión de al menos un compañero antes de hacer merge
```

V. Criterios de evaluación

Criterio	Indicadores	Puntaje
API funcional	Endpoints responden correctamente, se consume API externa, los datos persisten en BD.	25 pts
Docker	Dockerfile y docker-compose.yml correctos, contenedores levantan sin errores.	20 pts
Pipeline CI/CD	Workflow de GitHub Actions ejecuta pruebas y build automáticamente en cada push.	20 pts
Control de versiones	Historial de commits, uso de ramas, Pull Requests con revisión de pares.	15 pts
Documentación	README.md claro, APRENDIZAJES.md completo, .env.example presente.	10 pts
Presentación y Demo	Demo funcional con Postman, explicación clara de la arquitectura y el pipeline.	10 pts

Puntos Extra (+10 pts): Crea más de una API y haz que interactuen entre sí dentro de la arquitectura de contenedores.

VI. Guía de apoyo técnico

6.1 Estructura recomendada del repositorio

```
mi-api/
├── .github/
│   └── workflows/
│       └── ci.yml
├── src/                # código fuente de la API
├── tests/              # pruebas automatizadas
├── Dockerfile
├── docker-compose.yml
├── .env.example        # plantilla de variables de entorno
├── .gitignore          # incluye .env y node_modules/
├── README.md
└── APRENDIZAJES.md
```

6.2 Ejemplo mínimo de workflow CI/CD

Crea el archivo `.github/workflows/ci.yml` con el siguiente contenido como punto de partida:

```
name: CI Pipeline
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Instalar dependencias
        run: npm install
      - name: Ejecutar pruebas
        run: npm test
      - name: Construir imagen Docker
        run: docker build -t mi-api .
```

6.3 Buenas prácticas que se evalúan

- **Commits atómicos:** un commit = un cambio lógico, con mensaje descriptivo en imperativo (ej: 'Agregar endpoint de búsqueda').
- **Pull Requests pequeños y enfocados:** facilitan la revisión de código entre compañeros.
- **Variables de entorno:** nunca credenciales en el código; siempre leer desde `process.env` o equivalente.
- **Manejo de errores en la API:** responder con códigos HTTP correctos (400, 404, 500) y mensajes claros.
- **Pruebas significativas:** verificar que los endpoints responden con el formato y datos esperados.

VII. Ejemplos de ideas de proyectos

A continuación se presentan ideas concretas con sus componentes técnicos para orientar la elección:

Idea de API	API Externa	Base de Datos	Funcionalidad Extra
Sugerencias de restaurantes	Yelp API	MongoDB	Guardar búsquedas y favoritos
Clima extendido	OpenWeatherMap	MySQL	Guardar ubicaciones favoritas
Sala de chat + clima	OpenWeatherMap	MongoDB	Comandos de chat (clima Ciudad)
Rutas y puntos de interés	Google Maps Embed	MySQL	Historial de rutas consultadas

VIII. Entregas y fechas clave

Semana	Entregable	Forma de entrega
	README.md inicial + diseño de la API aprobado por el docente	Commit en el repositorio + exposición breve al grupo
	Código de la API con al menos 3 endpoints funcionales y PR por integrante	Repositorio en GitHub
	Dockerfile + docker-compose.yml funcionando localmente	Repositorio + video de demo de 2 min
	Pipeline CI/CD operando en GitHub Actions	Repositorio + captura del badge verde
	Presentación final + APRENDIZAJES.md + repositorio limpio	Exposición presencial + repositorio final

IX. Recomendaciones finales

Lista de verificación antes de la entrega final:

- ☐ El repositorio es público (o accesible para el docente)
- ☐ No hay archivos .env ni credenciales en el historial de commits
- ☐ El archivo README.md tiene instrucciones para correr el proyecto
- ☐ El APRENDIZAJES.md describe el proyecto y los aprendizajes del equipo
- ☐ Todos los contenedores levantan con: docker compose up --build
- ☐ El pipeline de GitHub Actions está en verde
- ☐ Cada integrante tiene al menos 4 commits significativos
- ☐ Se prepararon los endpoints en Postman para la demo

¡Éxito en el proyecto!

Recuerda que el objetivo no es solo que funcione, sino que sea software de calidad: legible, documentado, probado y desplegable automáticamente.